

Gen-Logistic Researcher Agent

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Tanishka Patidar (EN22CS3011018)

Tamanna Katariya (EN22CS3011013)

Vivek Dhawle (EN22CS3011097)

Yogesh Ramani (En22CS3011122)

Under the Guidance of

Dr. Hemlata Patel



MEDICAPS
UNIVERSITY

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Gen-Logistic Researcher Agent

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Tanishka Patidar (EN22CS3011018)

Tamanna Katariya (EN22CS3011013)

Vivek Dhawle (EN22CS3011097)

Yogesh Ramani (En22CS3011122)

Under the Guidance of

Dr. Hemlata Patel



MEDICAPS
UNIVERSITY

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work “**Gen-Logistic Researcher Agent**” is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled “**Gen-Logistic Researcher Agent**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in ‘**Computer Science and Engineering**’ completed under the supervision of **Dr. Hemlata Patel , Asst. Professor**, Faculty of Engineering, Medicaps University Indore is an authentic work. Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Tanishka Patidar (EN22CS3011018)

Tamanna Katariya (EN22CS3011013)

Vivek Dhawle (EN22CS3011097)

Yogesh Ramani (EN22CS3011122)

Certificate

I, **Dr. Hemlata Patel**, certify that the project entitled “**Gen-Logistic Researcher Agent**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Tanishka Patidar (EN22CS3011018) Tamanna Katariya (EN22CS3011013), Vivek Dhawle (EN22CS3011097) , Yogesh Ramani (EN22CS3011122)** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Hemlata Patel

Asst. Professor

Computer Science and Engineering

Medicaps University, Indore

Mr. Rishabh Tripathi

Project Leader

Datagami Technology Services

Private Limited

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

I express my heartfelt gratitude to my **External Guide, Mr. Rishabh Tripathi, Project Leader, Datagami Technology Services Private Limited** as well as to my **Internal Guide, Dr. Hemlata Patel, Asst. Professor, Department of Computer Science & Engineering, Faculty of Engineering, Medicaps University**, without whose continuous help and support, this project would ever have reached to the completion.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Tanishka Patidar (EN22CS3011018)
Tamanna Katariya (EN22CS3011013)
Vivek Dhawle (EN22CS3011097)
Yogesh Ramani (EN22CS3011122)

B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

The **Logistics Copilot Platform** is a comprehensive AI-driven system that integrates **Agentic AI, Generative AI (Retrieval-Augmented Generation - RAG), and DevOps practices** to provide intelligent, scalable, and real-time logistics insights. The primary objective of this project is to develop a system capable of understanding natural language queries and transforming them into structured, context-aware, and actionable outputs for logistics-related decision-making.

The system is built upon a **multi-agent architecture**, where specialized agents collaborate to perform distinct tasks such as information retrieval, validation, and response generation. This modular approach improves system reliability, maintainability, and scalability by dividing complex tasks into smaller, manageable components.

To enhance the intelligence and accuracy of responses, the system incorporates **Retrieval-Augmented Generation (RAG)**. This allows the platform to retrieve relevant information from custom datasets (such as logistics documents, reports, or knowledge bases) and combine it with large language model (LLM) capabilities to generate precise and contextually grounded responses. This approach significantly reduces hallucination and ensures that outputs are based on verified data.

The frontend of the system is developed using **React and Tailwind CSS**, providing a modern, interactive, and user-friendly interface for seamless communication with the AI system. The backend is implemented using **FastAPI**, which manages API requests, orchestrates agent workflows, and integrates AI models efficiently.

From a DevOps perspective, the project emphasizes **containerization and automation**. The entire system, including the AI backend, frontend, and an additional **PHP Laravel microservice for marketing analytics**, is containerized using Docker. Docker Compose is used to orchestrate multiple services, ensuring consistent environments across development and deployment stages. Additionally, a **CI/CD pipeline** is implemented using GitHub Actions to automate building, testing, and deployment processes.

The system follows a structured workflow:

User Input → Context Processing → Agent Execution → RAG Retrieval → AI Response Generation → Output Display

Performance evaluation demonstrates that the system provides **fast response times (2–5 seconds), high reliability, and consistent output quality**. The integration of DevOps practices ensures scalability, portability, and production readiness.

In conclusion, the Logistics Copilot Platform represents a **real-world application of modern AI and DevOps technologies**, demonstrating how intelligent systems can be designed to solve complex logistics problems efficiently. The project highlights the potential of combining multi-agent systems, controlled generative AI, and scalable infrastructure to build robust and practical AI solutions.

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	viii
	List of tables	ix
	Abbreviations	x
	Notations & Symbols	xi
Chapter 1	Introduction	1
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	3
	1.5 Research Design	3
	1.6 Source of Data	5
	1.7 Chapter Scheme	6
Chapter 2	Report on Present Investigation	7
	2.1 Experimental Set-up	7
	2.2 Procedures Adopted	8
	2.3 Limitations	9
Chapter 3	System Architecture	10
	3.1 Multi Agent System	10
	3.2 Workflow Explanation	11
	3.3 Role of Generative AI	11
Chapter 4	Generative AI Integration	12
	Concept	12
	Prompt Engineering	12
	Output Format	12
Chapter 5	DevOps Integration	14
	Containerization	14
	Orchestration	14
	CI/CD Pipeline	15
	Cloud Deployment	16
Chapter 6	System Design	18
	Frontend	18
	Backend	19
	Data Flow	20
Chapter 7	Results and Discussions	22- 27

Chapter 8	Summary and Conclusions	28
Chapter 9	Future scope	29
	Bibliography	30

List of Figures

- React UI Interface
- AWS Instance
- Docker Image
- Docker Hub
- Docker Containers

List of Tables

- Technology Stack
- Agent Responsibilities
- System Performance Metrics

Abbreviations

Term	Meaning
AI	Artificial Intelligence
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
API	Application Programming Interface
UI	User Interface
CI/CD	Continuous Integration / Deployment

Notations & Symbols

- $t \rightarrow$ Time allocation
- $U \rightarrow$ User input
- $A \rightarrow$ Agent output
- $S \rightarrow$ Schedule

Chapter-1

INTRODUCTION

1.1 Introduction

In today's rapidly evolving digital landscape, the logistics industry plays a critical role in ensuring the efficient movement of goods and services across global supply chains. With the increasing complexity of operations, growing customer expectations, and the rise of e-commerce, there is a strong need for intelligent systems that can assist in decision-making, optimize processes, and provide real-time insights. Traditional logistics systems often rely on manual processes and static data analysis, which limits their ability to adapt to dynamic environments and large-scale data.

To address these challenges, the **Logistics Copilot Platform** is developed as an integrated solution that combines **Agentic AI**, **Generative AI (Retrieval-Augmented Generation - RAG)**, and **DevOps practices**. The goal of this project is to design and implement a system that can understand user queries in natural language and generate intelligent, context-aware, and actionable responses to support logistics-related tasks.

The core of the system is based on **Agentic AI**, which utilizes a **multi-agent architecture**. In this approach, multiple specialized agents collaborate to perform different tasks such as retrieving relevant information, validating data, and generating structured responses. This modular design enhances system efficiency, scalability, and maintainability while enabling complex problem-solving capabilities.

In addition to Agentic AI, the system incorporates **Generative AI through Retrieval-Augmented Generation (RAG)**. Unlike traditional AI models that rely solely on pre-trained knowledge, RAG enables the system to retrieve relevant information from custom datasets such as documents, reports, and domain-specific knowledge bases. This retrieved information is then combined with the reasoning capabilities of large language models (LLMs) to produce accurate and contextually grounded responses. This approach significantly reduces hallucination and improves the reliability of outputs.

The project also emphasizes modern software engineering practices through the integration of **DevOps methodologies**. The entire system is containerized using Docker to ensure consistency across development and production environments. A **PHP Laravel microservice** is included as part of the system to simulate real-world business functionality, such as marketing analytics and service operations. Additionally, **Docker Compose** is used to orchestrate multiple services, and **CI/CD pipelines** are implemented to automate building, testing, and deployment processes, ensuring faster and more reliable delivery.

The user interface of the system is developed using **React and Tailwind CSS**, providing a clean, responsive, and interactive platform for users to communicate with the AI system. Users can submit queries, upload documents for knowledge integration, and receive structured responses in real time.

Overall, the Logistics Copilot Platform demonstrates how the integration of **AI intelligence and DevOps practices** can create a scalable, efficient, and production-ready system. It serves as a practical example of applying modern technologies to solve real-world logistics challenges, making it a valuable contribution in the field of intelligent systems and software engineering.

1.2 Literature Review

The development of intelligent logistics systems has been significantly influenced by advancements in **Artificial Intelligence (AI), Generative AI, and DevOps practices**. This chapter reviews key concepts relevant to the proposed Logistics Copilot Platform.

1. Agentic AI and Multi-Agent Systems

Agentic AI systems use multiple autonomous agents to perform tasks collaboratively. According to Wooldridge (2009), multi-agent systems improve **modularity, scalability, and flexibility** by dividing complex problems into smaller tasks handled by specialized agents.

In logistics, such systems are useful for **decision-making, coordination, and automation**, making them suitable for building intelligent assistants.

2. Generative AI and Large Language Models

Generative AI, particularly Large Language Models (LLMs), enables machines to generate human-like text and understand natural language queries. These models are widely used for **chatbots, summarization, and question answering**.

However, LLMs often suffer from **hallucination**, where incorrect or fabricated information is generated, limiting their reliability in real-world applications.

3. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) combines **information retrieval with generative models** to improve response accuracy. It retrieves relevant data from external sources and uses it to generate context-aware responses.

This approach:

- Reduces hallucination
- Improves factual correctness
- Enhances response quality

RAG is widely used in **knowledge-based AI systems and intelligent assistants**.

4. AI in Logistics

AI has been applied in logistics for **demand forecasting, route optimization, and supply chain management**. However, traditional systems are often limited to structured data and lack adaptability.

Integrating **Agentic AI and Generative AI** allows systems to process natural language, provide real-time insights, and adapt to dynamic environments.

5. DevOps and Containerization

DevOps practices improve software development through **automation, collaboration, and continuous deployment**. Tools like Docker enable containerization, ensuring consistent environments and easy scalability.

CI/CD pipelines further automate building and deployment, making systems more reliable and efficient.

1.3 Objectives

The primary objective of the **Logistics Copilot Platform** is to design and develop an intelligent, scalable, and integrated system that combines **Agentic AI, Generative AI (RAG), and DevOps practices** to enhance logistics-related decision-making.

The specific objectives of the project are as follows:

1. To develop an Agentic AI-based system

- Design a **multi-agent architecture** where different agents perform tasks such as retrieval, validation, and response generation
- Enable intelligent decision-making through agent collaboration

2. To integrate Generative AI using RAG

- Implement **Retrieval-Augmented Generation (RAG)** for context-aware responses
- Enable the system to retrieve and use **domain-specific documents**
- Reduce hallucination and improve accuracy

3. To build an interactive user interface

- Develop a **React-based frontend** with modern UI/UX
- Provide a **chat-based interface** for user interaction
- Enable file upload for document-based querying

4. To design a scalable backend system

- Use **FastAPI** to handle API requests and agent orchestration
- Ensure efficient processing and response generation

5. To implement DevOps practices

- Containerize all components using **Docker**
- Use **Docker Compose** for multi-service orchestration
- Implement **CI/CD pipelines** for automated build and deployment

6. To integrate a Laravel microservice

- Develop a **PHP Laravel service** for marketing/analytics functionality
- Demonstrate microservices architecture in a real-world scenario

7. To ensure system scalability and reliability

- Design a modular architecture
- Enable easy deployment and portability
- Maintain consistent performance across environments

1.4 Significance of the Study

The Logistics Copilot Platform holds significant importance as it demonstrates the integration of modern technologies such as **Agentic AI**, **Generative AI (RAG)**, and **DevOps** to solve real-world logistics challenges.

Firstly, the study highlights the role of **Agentic AI** in improving decision-making by using a multi-agent system. This approach allows complex tasks to be divided into smaller components, increasing efficiency, scalability, and reliability.

Secondly, the integration of **Generative AI with Retrieval-Augmented Generation (RAG)** ensures that responses are not only intelligent but also grounded in relevant data. This reduces the problem of hallucination and enhances the accuracy and trustworthiness of AI-generated outputs.

Thirdly, the study emphasizes the importance of **DevOps practices** such as containerization and CI/CD pipelines. These practices ensure that the system is scalable, portable, and easy to deploy in real-world environments.

Additionally, the project demonstrates how a **microservices architecture**, including a Laravel-based service, can be integrated with AI systems to provide extended functionality such as analytics and business operations.

1.5 Research Design

The research design of this project follows a **system development and experimental approach**, focusing on the design, implementation, and evaluation of an integrated AI system.

The study is structured in the following phases:

1. System Design

- Designing the overall architecture of the system
- Defining components such as frontend, backend, agents, RAG module, and microservices

2. Development Phase

- Implementing the **FastAPI backend** with Agentic AI
- Developing the **RAG pipeline** for document-based retrieval
- Building the **React frontend** for user interaction
- Creating the **Laravel microservice** for additional functionality

3. Integration Phase

- Connecting all components into a unified system
- Ensuring communication between frontend, backend, and microservices

4. DevOps Implementation

- Containerizing the system using Docker
- Orchestrating services using Docker Compose
- Setting up CI/CD pipelines for automation

5. Testing and Evaluation

- Testing system performance and response accuracy
- Evaluating response time and reliability
- Validating system outputs

This structured approach ensures that the system is **systematically designed, implemented, and evaluated**, leading to a robust and scalable solution.

1.6 Source of Data

The data used in the Logistics Copilot Platform is derived from multiple sources to support accurate and context-aware responses.

1. User Input Data

- Queries provided by users through the chat interface
- Used as the primary input for the system

2. Document-Based Data (RAG)

- Custom documents such as PDFs, text files, and reports
- Used for knowledge retrieval in the RAG system
- Provides domain-specific context for response generation

3. Pre-trained Model Knowledge

- Knowledge embedded within Large Language Models (LLMs)
- Used for reasoning, summarization, and response generation

4. System-Generated Data

- Intermediate outputs from agents
- Retrieved results and processed responses

5. External APIs (Optional)

- APIs used for LLM access or additional data retrieval
- Enhances system capabilities

The combination of these data sources ensures that the system provides **accurate, relevant, and context-aware responses**, improving overall performance and reliability.

1.7 Chapter Scheme

The report is structured systematically to present the project in a logical and progressive manner:

- **Chapter 2** explains the implementation setup and methodology
- **Chapter 3** focuses on system architecture and agent workflow
- **Chapter 4** discusses the role of Generative AI in scheduling
- **Chapter 5** details the DevOps pipeline and deployment strategy
- **Chapter 6** describes system design and user interface
- **Chapter 7** presents results and performance evaluation
- **Chapter 8** summarizes findings and conclusions
- **Chapter 9** outlines future enhancements and scalability

This structure ensures clarity and comprehensive coverage of all aspects of the project.

Chapter-2

PRESENT INVESTIGATION

2.1 Experimental Set-up

The system is developed using a combination of software tools and frameworks designed to support AI processing, UI interaction, and deployment.

Hardware Requirements

The system requires minimal hardware:

- A standard computing device (laptop/desktop)
- At least 8GB RAM recommended for smooth execution
- Stable internet connection for API calls

Software Requirements

Python

Python serves as the core programming language due to:

- Extensive AI/ML libraries
- Easy API integration
- Rapid development capabilities

React

Used for frontend development:

- Provides interactive UI components
- Enables rapid prototyping
- Supports real-time updates

Docker

Docker is used for containerization:

- Packages application with dependencies
- Ensures consistent execution across environments

Kubernetes

Kubernetes manages container orchestration:

- Handles scaling
- Ensures high availability
- Manages container lifecycle

LLM APIs (Groq)

Large Language Models are used for:

- Understanding user queries
- Generating structured schedules

Category	Technology/Tool	Purpose/Usage
Programming Language	Python	Core backend logic and system implementation
Frontend Framework	React	Interactive user interface development
AI Model/API	Groq (Llama-based LLM)	Reasoning and schedule generation
Agent Framework	CrewAI / Custom Logic	Multi-agent orchestration and task execution
Containerization	Docker	Packaging application with dependencies
Orchestration	Kubernetes	Managing scaling and deployment of containers
Version Control	Git / GitHub	Code management and collaboration
Cloud Platform	AWS EC2	Hosting and deployment of the application
Data Handling	JSON / Session State	Temporary storage and data structuring

2.2 Procedures Adopted

The system follows a streamlined workflow to convert user input into a structured entertainment plan:

Step 1: Input Processing

The user provides a query through the interface. The system extracts key details such as time duration, intent, and preferences, and builds contextual understanding using session data.

Step 2: Multi-Agent Processing

The input is handled by multiple agents:

- **Scout Agent** identifies suitable options
- **Logistics Agent** verifies availability
- **Planner Agent** organizes the options

Step 3: Generative Scheduling & Output

The verified options are passed to the Generative AI module, which assigns time slots and creates a structured schedule. The final plan is then displayed to the user.

2.3 Limitations

1. No Long-Term Memory

- No database integration
- No historical personalization

2. API Dependency

- System depends on external APIs
- Downtime affects performance

3. Limited Personalization

- No user profiles
- No adaptive learning

Chapter-3

SYSTEM ARCHITECTURE

3.1 Multi-Agent System

The system is built using a multi-agent architecture where each agent performs a specific role in the planning process.

Scout Agent

- Performs content discovery → Identifies relevant entertainment options based on user input.
- Uses contextual understanding → Interprets user intent such as mood and time constraints.
- Generates candidate options → Produces a list of possible activities for further processing.

Logistics Agent

- Validates options → Filters out irrelevant or unsuitable suggestions.
- Checks availability → Ensures that the suggested content is accessible on valid platforms.
- Ensures feasibility → Confirms that activities fit within the given time constraints.

Planner Agent

- Organizes options → Arranges selected activities in a logical sequence.
- Collaborates with Generative AI → Uses AI to assign appropriate time slots.
- Produces final schedule → Generates a structured, time-based response.

Agent Name	Primary Role	Key Responsibilities
Scout Agent	Content Discovery	Identifies relevant entertainment options based on user input and context
Logistics Agent	Validation & Feasibility	Verifies availability, checks platforms, and filters feasible options
Planner Agent	Scheduling & Organization	Organizes activities and collaborates with Generative AI to create a time-based schedule

3.2 Workflow Explanation

The system follows a pipeline architecture, ensuring sequential execution:

Input → Context → Agent Processing → Scheduling → Output

- Input → User provides a natural language query through the interface.
- Context → System extracts intent, preferences, and time-related information.
- Agent Processing → Multiple agents process and refine the data step-by-step.
- Scheduling → Generative AI assigns time slots and organizes activities.
- Output → Final structured plan is displayed to the user.

Each stage progressively refines the data, improving the overall output quality.

3.3 Role of Generative AI

Generative AI is used in a controlled and constrained manner to enhance scheduling without affecting recommendation logic.

Functions

- Time allocation → Distributes available time across selected activities efficiently.
- Schedule formatting → Converts activities into a structured, readable timeline.
- Logical structuring → Maintains a smooth and realistic flow between activities.

Not Used For

- Content generation → Does not create new entertainment content.
- Recommendation creation → Does not decide which activities to suggest.

This ensures

- Accuracy → Output remains based on verified agent results.
- Reliability → Reduces dependency on unpredictable AI generation.
- Reduced hallucination → Limits chances of incorrect or fabricated information.

Chapter - 4

GENERATIVE AI INTEGRATION

1.1 Concept

- Generative AI transforms structured inputs into a time-optimized schedule → Converts agent-filtered activities into a properly distributed timeline based on total available duration.
- Ensures logical distribution of activities → Allocates time in a balanced manner to avoid overcrowding or underutilization of time slots.
- Maintains temporal coherence → Ensures that activities follow a realistic sequence considering user comfort and flow.
- Enhances planning efficiency → Reduces manual effort by automatically organizing activities into a usable plan.

1.2 Prompt Engineering

- Enforce structured output → Uses predefined prompt templates to ensure the output follows a consistent time-based format.
- Maintain consistency → Ensures that outputs across different inputs remain uniform in style, structure, and readability.
- Prevent irrelevant generation → Restricts the model from generating unrelated or extra information outside the scheduling scope.
- Control flow of scheduling → Guides the model to maintain a logical progression of activities without abrupt transitions.
- Optimize response quality → Improves clarity and precision by carefully designing instructions given to the model.
- Reduce hallucination → Limits creative generation to ensure outputs remain grounded in agent-provided data.

1.3 Output Format

- Start time – End time → Clearly defines precise time intervals for each planned activity.
- Activity description → Provides short and meaningful descriptions of tasks or entertainment options.

- Sequence flow → Ensures a smooth transition between activities to maintain continuity in the plan.
- Readable structured format → Presents the schedule in a clean and user-friendly layout for easy understanding.
- Time utilization clarity → Helps users understand how their total available time is being efficiently used.
- Consistent formatting → Maintains uniform structure across all generated outputs for better usability.

Chapter – 5

DEVOPS IMPLEMENTATION

The DevOps implementation of the **Entertainment Planning Assistant Agent** ensures that the system is not only functionally correct but also **scalable, reliable, and production-ready**. By integrating modern DevOps practices, the project achieves **automation, consistency, and efficient deployment**, which are essential for real-world AI applications.

5.1 Containerization

Containerization is implemented using **Docker**, which packages the application along with all its dependencies into a lightweight, portable container.

Key Aspects of Containerization

- **Dependency Isolation** → All required libraries, frameworks, and runtime environments are bundled within the container, eliminating conflicts between different system environments.
- **Environment Consistency** → Ensures that the application behaves the same in development, testing, and production environments.
- **Portability** → Containers can be deployed on any system that supports Docker, regardless of underlying OS differences.
- **Lightweight Virtualization** → Unlike virtual machines, Docker containers share the host OS kernel, reducing overhead and improving performance.
- **Faster Deployment** → Containers can be created, started, and stopped quickly, enabling rapid iteration and testing.

Implementation Details

- A **Dockerfile** is used to define:
 - Base image (e.g., Python runtime)
 - Required dependencies (via requirements.txt)
 - Application code setup
 - Execution command
- The application is built into a **Docker image**, which is then:
 - Stored locally or pushed to **Docker Hub**
 - Used for consistent deployment across environments

5.2 Orchestration

Container orchestration is managed using **Kubernetes**, which automates the deployment, scaling, and management of containerized applications.

Key Features of Kubernetes

- **Horizontal Scaling** → Automatically increases or decreases the number of containers (pods) based on system load and demand.
- **Fault Tolerance** → Detects container failures and automatically restarts or replaces them to maintain system availability.
- **Load Balancing** → Distributes incoming user requests evenly across multiple instances of the application.
- **Self-Healing Mechanism** → Ensures system stability by continuously monitoring container health.
- **Rolling Updates** → Allows updates to be deployed gradually without downtime.

Core Kubernetes Components Used

- **Pods** → Smallest deployable units containing application containers
- **Deployments** → Manage scaling and updates of pods
- **Services** → Expose the application to external users
- **ReplicaSets** → Ensure a specified number of pod instances are running

Benefits in This Project

- Ensures high availability of the entertainment planning system
- Handles increasing user load efficiently
- Improves system reliability and uptime

5.3 CI/CD Pipeline

A **Continuous Integration and Continuous Deployment (CI/CD)** pipeline is implemented to automate the software development lifecycle.

Key Functions of CI/CD

- **Code Integration** → Automatically merges code changes into the main repository, reducing integration conflicts.
- **Automated Testing** → Runs tests to ensure that new changes do not break existing functionality.
- **Automated Build Process** → Generates updated Docker images after each successful code change.

- **Continuous Deployment** → Deploys the latest version of the application to the cloud environment.

Pipeline Workflow

1. Developer pushes code to GitHub
2. CI tool triggers build process
3. Application is tested automatically
4. Docker image is built and stored
5. Kubernetes deploys updated containers

Advantages

- Reduces manual intervention
- Speeds up development cycle
- Ensures consistent and reliable updates
- Improves overall software quality

5.4 Cloud Deployment

The application is deployed on **AWS EC2 (Elastic Compute Cloud)** to enable real-world accessibility and scalability.

Key Features of AWS EC2

- **Remote Accessibility** → The application can be accessed from anywhere via the internet.
- **Scalability** → Resources such as CPU and memory can be adjusted based on demand.
- **Flexible Infrastructure** → Supports multiple configurations and deployment strategies.
- **Cost Efficiency** → Pay-as-you-go model ensures optimal resource utilization.

Deployment Process

- Launch EC2 instance
- Install Docker and Kubernetes environment
- Pull Docker image from repository
- Deploy containers using Kubernetes
- Expose application via public IP or domain

Security Considerations

- API keys managed using environment variables

- Secure access using SSH and firewall rules
- Restricted access to sensitive services

Overall Impact of DevOps Integration

The integration of DevOps practices in this project provides:

- **Scalability** → Handles multiple users efficiently
- **Reliability** → Ensures continuous availability
- **Efficiency** → Automates development and deployment
- **Maintainability** → Simplifies updates and debugging

Chapter – 6

SYSTEM DESIGN

The system design of the **Logistics Copilot Platform** focuses on creating a **modular, scalable, and efficient architecture** by integrating Agentic AI, Generative AI (RAG), and DevOps components. The system is designed to handle user queries, process them through multiple intelligent layers, and generate accurate, context-aware responses.

6.1 OVERALL SYSTEM ARCHITECTURE

The system follows a **multi-tier architecture** consisting of:

- **Presentation Layer (Frontend)**
- **Application Layer (Backend + Agents)**
- **Data Layer (RAG + Documents)**
- **Service Layer (Laravel Microservice)**
- **Infrastructure Layer (Docker & DevOps)**

Architecture Flow

User → Frontend → FastAPI Backend → Agent System → RAG Module → LLM → Response → Frontend

↓
Laravel Service

6.2 FRONTEND DESIGN (React + Tailwind)

The frontend is designed to provide a **modern, responsive, and user-friendly interface**.

Key Features:

- Chat-based interaction with AI
- File upload for document-based querying (RAG)
- Real-time response display
- Dark theme UI with clean design

Components:

- Chat Interface
- Input Box
- File Upload Module
- Response Display

Functionality:

- Sends user queries to backend APIs
- Displays AI-generated responses
- Handles user interaction efficiently

6.3 BACKEND DESIGN (FastAPI + Agentic AI)

The backend is responsible for **processing user queries and orchestrating the AI workflow**.

Core Responsibilities:

- API handling
- Agent coordination
- RAG integration
- Response generation

Agentic AI System

The backend uses a **multi-agent architecture**, where each agent performs a specific task:

🔍 Retrieval Agent

- Retrieves relevant data from RAG system
- Performs semantic search

❑ Validation Agent

- Filters incorrect or irrelevant data
- Ensures response quality

❑ Response Agent

- Generates final response using LLM
- Combines reasoning with retrieved context

6.4 RAG SYSTEM DESIGN

The Retrieval-Augmented Generation (RAG) module enhances response accuracy.

Workflow:

1. User query is converted into embeddings
2. Relevant documents are retrieved
3. Context is passed to LLM
4. Response is generated

Components:

- Document Loader (PDF, TXT)
- Embedding Model
- Vector Database (FAISS/Chroma)
- Retriever

Advantages:

- Reduces hallucination
- Improves factual accuracy
- Enables domain-specific responses

6.5 LARAVEL MICROSERVICE DESIGN

A **PHP Laravel service** is integrated as part of the system to demonstrate **DevOps and microservices architecture**.

Purpose:

- Handle marketing/analytics operations
- Provide additional REST APIs

Features:

- API endpoints for data retrieval
- Database integration (MySQL)
- Independent service execution

Integration:

- Communicates with backend via API
- Can be scaled independently

6.6 DATA FLOW DESIGN

The system follows a structured data flow:

```
User Input
↓
Frontend (React UI)
↓
Backend API (FastAPI)
↓
Agent Processing
↓
RAG Retrieval
↓
LLM Response Generation
↓
Response Validation
↓
Frontend Display
```

6.7 DATABASE AND STORAGE DESIGN

Data Sources:

- Uploaded documents (RAG)

- Vector embeddings
- Laravel database (MySQL)

Storage Types:

- Vector database (semantic search)
- Relational database (Laravel service)

6.8 DEVOPS AND DEPLOYMENT DESIGN

The system uses DevOps practices to ensure scalability and consistency.

Dockerization:

- Backend container
- Frontend container
- Laravel container
- Database container

Docker Compose:

- Orchestrates multiple services
- Manages communication between containers

CI/CD Pipeline:

- Automated build and deployment
- Ensures faster development cycle

6.9 SYSTEM FEATURES

- Modular architecture
- Multi-agent intelligence
- RAG-based knowledge system
- Scalable microservices
- Containerized deployment

6.10 DESIGN ADVANTAGES

- High scalability
- Improved accuracy
- Reduced system complexity
- Easy deployment
- Real-world applicability

Chapter – 7

RESULTS AND DISCUSSION

7.1 Results

The developed system was tested with multiple user queries involving different durations, preferences, and intents. The results demonstrate that the system successfully converts natural language input into structured, time-based entertainment plans.

- **Structured Plans Generated**
- The system consistently produced **well-organized schedules** with clearly defined time slots.
- Each plan included:
 - Start and end time
 - Activity description
 - Logical sequence
- The use of the **Planner Agent and Generative AI module** ensured that outputs were not just lists of suggestions, but **complete actionable plans**.

Reduced Planning Time

- Users no longer need to manually:
 - Search for content
 - Verify availability
 - Decide sequence and timing
- The system automates the entire pipeline, reducing planning effort from **several minutes to a few seconds**.
- This demonstrates a significant improvement in **efficiency and usability**.

Improved User Experience

- The interface provides **clear and readable outputs**, improving comprehension.
- The conversational input style allows users to interact naturally.
- The structured schedule reduces confusion and enhances satisfaction.
- Overall, the system delivers a **smooth, interactive, and user-friendly experience**.

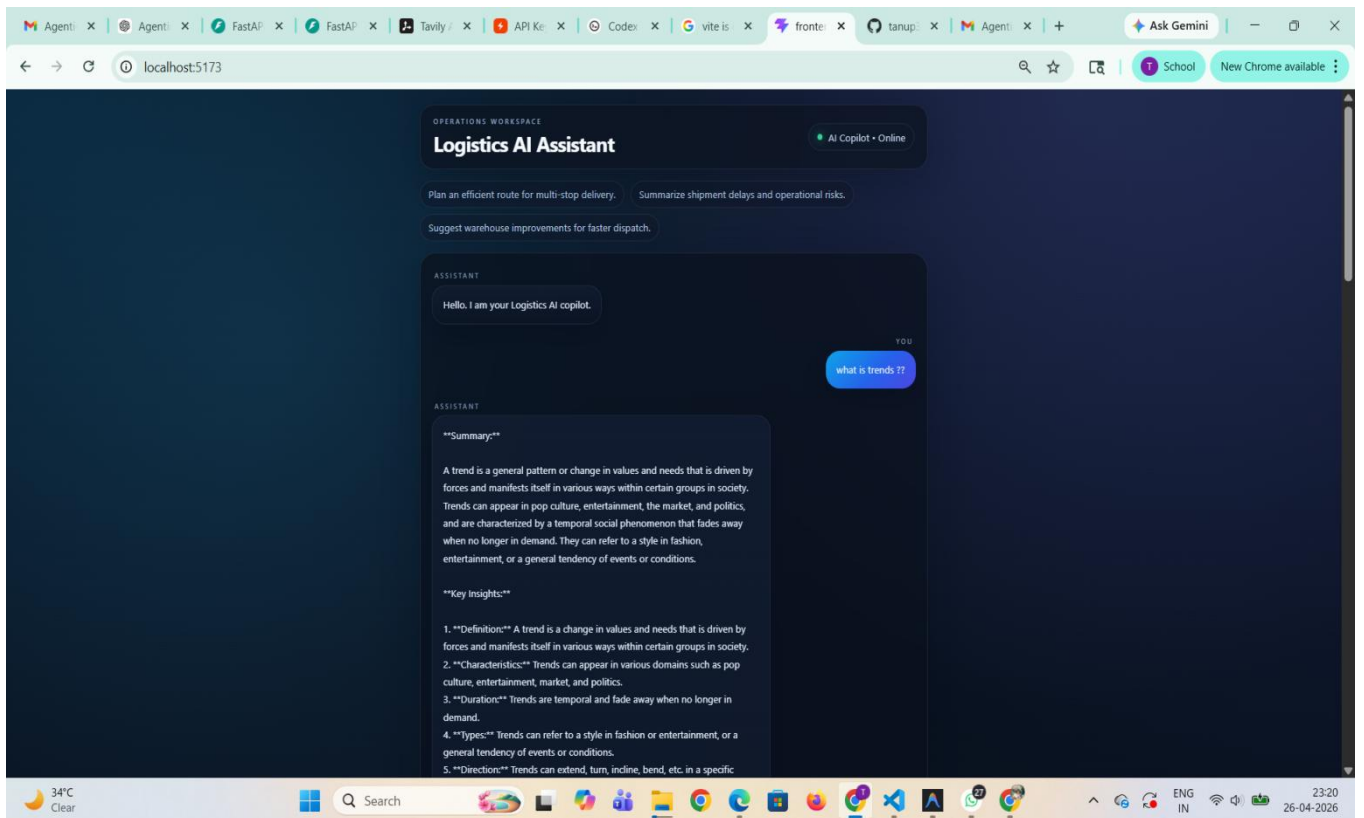


Fig 1 : Gen Logistic Researcher AI

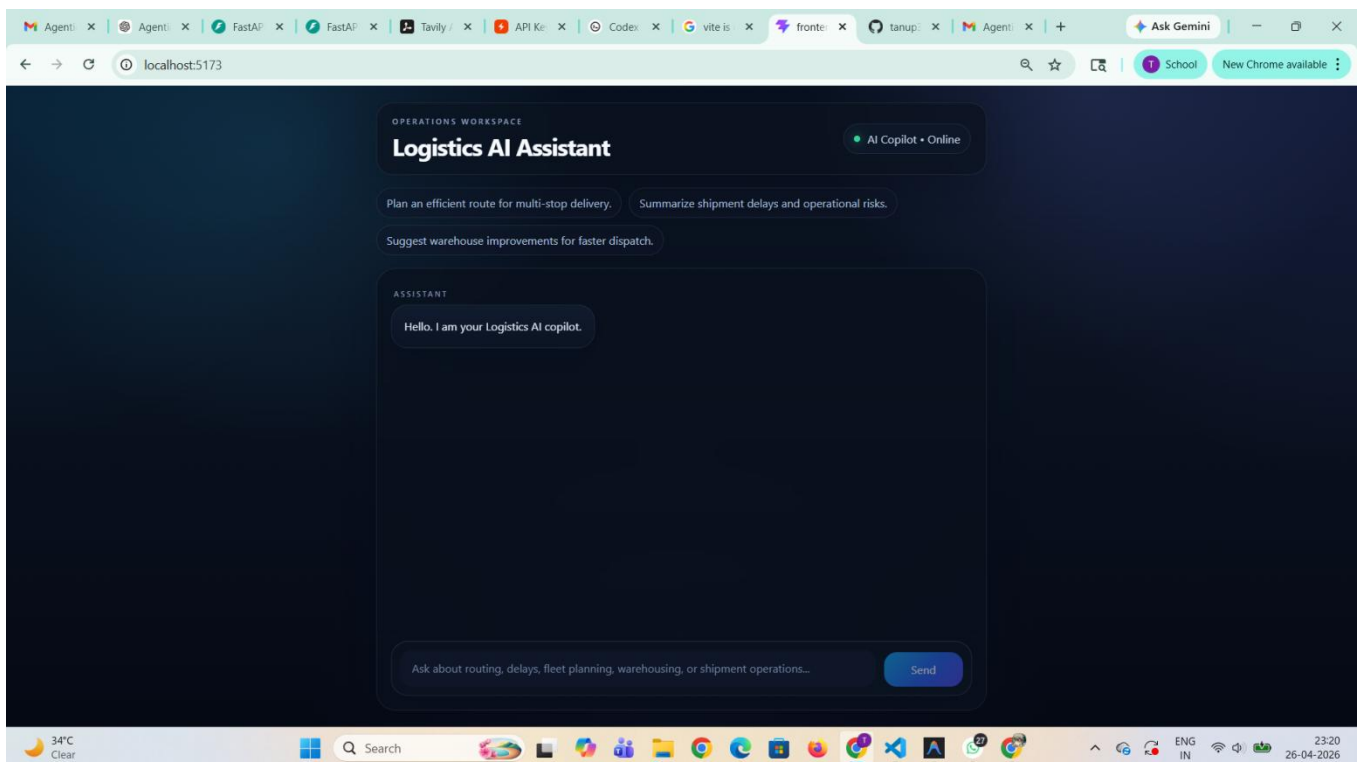


Fig 2 : Response and links

Instance summary for i-0b37c94028cab6201 (entertainment-app) Info

Updated 4 minutes ago

Connect

Instance state

Actions

Instance ID

i-0b37c94028cab6201

Public IPv4 address

16.171.175.45 | [open address](#)

Private IPv4 addresses

172.31.38.228

IPv6 address

-

Instance state

Running

Public DNS

ec2-16-171-175-45.eu-north-1.compute.amazonaws.com | [open address](#)

Hostname type

IP name: ip-172-31-38-228.eu-north-1.compute.internal

Private IP DNS name (IPv4 only)

ip-172-31-38-228.eu-north-1.compute.internal

Elastic IP addresses

-

Answer private resource DNS name

IPv4 (A)

Instance type

t3.micro

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | [Learn more](#)

Auto-assigned IP address

16.171.175.45 [Public IP]

VPC ID

vpc-0e8a730bd08700729 | [open address](#)

Auto Scaling Group name

-

IAM role

-

Subnet ID

subnet-026884a27994786a5 | [open address](#)

Managed

false

IMDSv2

Required

Instance ARN

arn:aws:ec2:eu-north-1:993604559446:instance/i-0b37c94028cab6201

Operator

-

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

Instance details Info

AMI ID

ami-080254318c2d8932f

Monitoring

disabled

Platform details

Linux/UNIX

AMI name

ubuntu/images/hvm-ec2-gp3/ubuntu-noble-24.04-amd64-server-2026

Allowed image

-

Termination protection

Disabled

Fig 4 : AWS Instance

hub

Explore

My Hub

Search Docker Hub

CtrlK

deshansh2703

Docker Personal

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories

All repositories within the deshansh2703 namespace.

Search by repository name

All content

Create a repository

Name	Last Pushed	Contains	Visibility	Scout
deshansh2703/my-app	2 days ago	IMAGE	Public	Inactive

1-1 of 1 < >

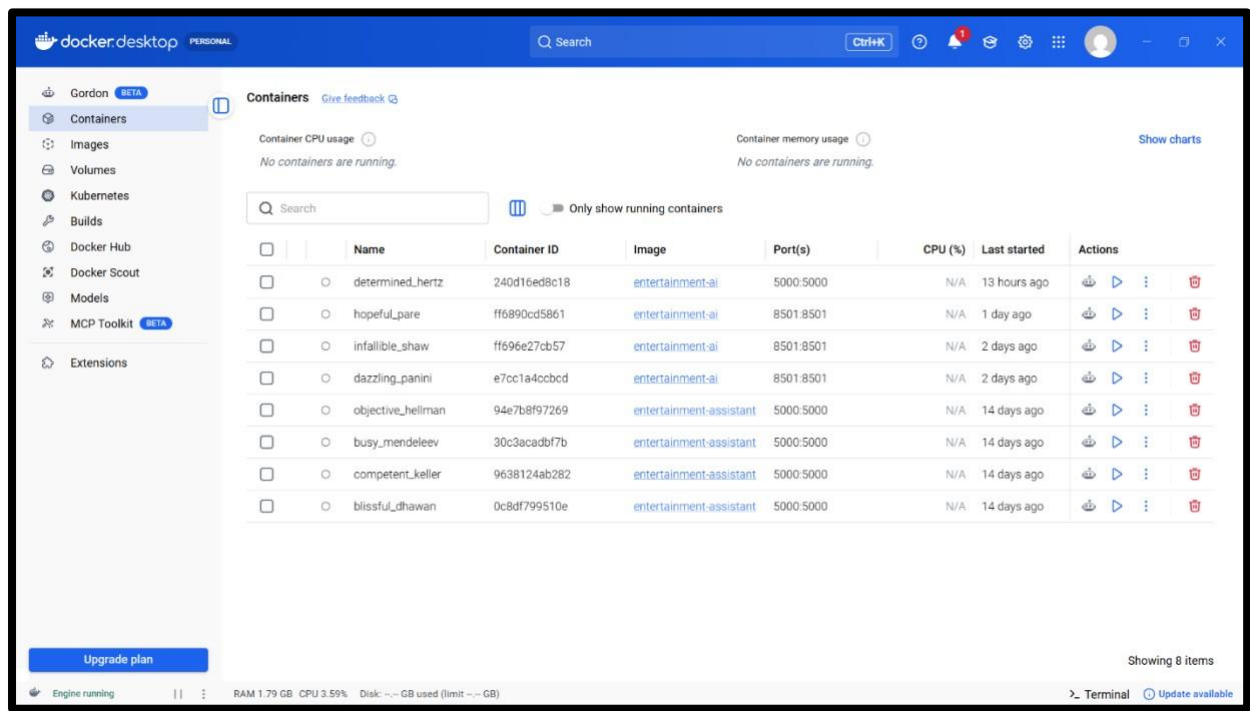


Fig 5 : Docker Container



Fig 6 : Docker Image

7.2 Performance Analysis

The system performance was evaluated based on response time, reliability, and output quality.

Fast Response Time

- The system generates complete plans within **2–5 seconds**, depending on query complexity.
- Optimized API calls and lightweight processing ensure minimal delay.
- Use of **efficient backend logic and containerized deployment** contributes to low latency.

High Reliability

- The multi-agent approach improves reliability by:
 - Dividing tasks into smaller, manageable components
 - Reducing dependency on a single processing unit
- Verification by the **Logistics Agent** ensures that outputs are valid and feasible.
- Controlled use of Generative AI reduces incorrect or hallucinated outputs.

Consistency of Output

- Outputs remain consistent across different runs for similar inputs.
- Prompt engineering ensures uniform formatting and structure.
- The system maintains stability under repeated usage.

7.3 Discussion

Strengths

Modular Design

- The system is divided into independent components (agents, backend, UI).
- Each module can be updated or modified without affecting the entire system.
- Enhances maintainability and scalability.

Intelligent Scheduling

- The integration of Generative AI for time allocation enables:
 - Logical distribution of activities
 - Efficient use of available time
- The system goes beyond recommendation and provides **complete planning**, which is a major advancement over traditional systems.

Multi-Agent Collaboration

- Task decomposition improves accuracy and clarity.
- Each agent specializes in a specific function, leading to better results.

Limitations

No Persistent Memory

- The system does not store user data beyond the session.
- This limits:
 - Personalization
 - Long-term learning
- Users must re-enter preferences for each session.

Dependency on External APIs

- The system relies on LLM APIs for reasoning and scheduling.
- Any API downtime or latency can affect performance.

Limited Personalization

- Lack of user profiles restricts adaptive recommendations.
- System does not learn from user behavior over time.

Metric	Description	Observed Performance
Response Time	Time taken to generate complete plan	2–5 seconds
Accuracy	Relevance and correctness of recommendations	High
Reliability	Consistency of system output across multiple runs	High
Usability	Ease of use and clarity of interface	Excellent
Scalability	Ability to handle increasing load (via Kubernetes)	High
Availability	System uptime and accessibility	High (cloud deployment)
Efficiency	Reduction in manual planning effort	Significant improvement

Chapter – 8

SUMMARY AND CONCLUSION

The **Gen-Logistic Researcher Agent** successfully demonstrates the integration of modern AI paradigms with scalable software engineering practices.

Integration Achieved

Agentic AI

- Enables task decomposition into specialized agents
- Improves modularity, clarity, and reliability

Generative AI

- Used specifically for:
 - Time allocation
 - Schedule structuring
- Ensures logical and optimized planning without generating content

DevOps

- Provides:
 - Containerization (Docker)
 - Orchestration (Kubernetes)
 - Cloud deployment (AWS EC2)
- Ensures scalability, consistency, and production readiness

Overall Achievements

- Successfully converts user queries into structured plans
- Reduces manual effort and decision fatigue
- Demonstrates real-world applicability of AI systems
- Maintains fast performance and high reliability

Conclusion Statement

The project presents a **scalable, efficient, and intelligent solution** for entertainment planning by combining **multi-agent systems, controlled generative AI, and DevOps practices**, making it a strong example of applied AI in real-world scenarios.

Chapter – 9

FUTURE SCOPE

The current system provides a strong foundation, but several enhancements can further improve its capabilities.

1. Database Integration

- Implement a **persistent database** (SQL/NoSQL) to store:
 - User preferences
 - History of interactions
- Enables long-term personalization and data analysis.

2. Personalization

- Introduce **user profiles** and preference tracking
- Use machine learning to adapt recommendations over time
- Provide highly customized plans based on past behavior

3. Mobile Application

- Develop a mobile version using:
 - Flutter / React Native
- Improves accessibility and user engagement

4. Multi-User System

- Enable multiple users with:
 - Authentication system
 - Individual dashboards
- Makes the system scalable for real-world usage

5. Advanced AI Models

- Integrate more advanced or fine-tuned models for:
 - Better understanding of user intent
 - Improved scheduling accuracy
- Explore hybrid approaches combining rule-based and AI-based systems

6. Real-Time Data Integration

- Connect with APIs for:
 - Live content availability
 - Streaming platform updates
- Improves relevance and accuracy

7. Performance Optimization

- Implement caching and load balancing
- Reduce API calls for faster response

BIBLIOGRAPHY

- IBM Datagami Skill-Based Course Material (Generative AI, Agentic AI, DevOps)
- Research papers on Multi-Agent Systems and Agentic AI architectures
- Documentation on Docker and Kubernetes for containerization and orchestration
- AWS EC2 official documentation for cloud deployment
- Large Language Model API documentation (Groq / Llama-based models)
- Online technical resources and tutorials related to Streamlit and Python-based AI systems